

network_recovery_manager.py 파일 정리

네트워크 장애 구간 기록 및 복구 영상 요청 모듈

전체 코드

```
# network_recovery_manager.py

import os
import re
from datetime import datetime
from urllib.parse import unquote

class NetworkRecoveryManager:
    def __init__(
        self,
        camera_id="cam01",
        server_url="http://라즈베리파이IP:8002/recover",
        recovery_dir="",
        min_failure_seconds=2.0,
        request_timeout=5,
    ):
        self.camera_id = camera_id
        self.server_url = server_url
        self.recovery_dir = recovery_dir or os.path.join(os.getcwd(), "복구 영상")
        self.min_failure_seconds = min_failure_seconds
        self.request_timeout = request_timeout

        self.failure_start_time = None
        self.requested_ranges = set()

    def has_active_failure(self):
        return self.failure_start_time is not None

    def record_failure(self, failed_time=None):
        failed_time = failed_time or datetime.now()

        if self.failure_start_time is None:
            self.failure_start_time = failed_time
            return {
                "started": True,
                "failure_start_time": self._format_time(self.failure_start_time),
            }

        return {
            "started": False,
            "failure_start_time": self._format_time(self.failure_start_time),
        }

    def record_recovery(self, recovered_time=None):
        if self.failure_start_time is None:
            return {
                "requested": False,
                "success": False,
                "reason": "no_active_failure",
            }

        recovered_time = recovered_time or datetime.now()
        failure_start_time = self.failure_start_time
        self.failure_start_time = None

        duration_seconds = (recovered_time - failure_start_time).total_seconds()
        payload = self.build_payload(failure_start_time, recovered_time)

        if duration_seconds < self.min_failure_seconds:
            return {
                "requested": False,
                "success": True,
                "skipped": True,
                "reason": "too_short",
                "duration_seconds": duration_seconds,
                "payload": payload,
            }

        request_key = self._get_request_key(payload)
        if request_key in self.requested_ranges:
            return {
                "requested": False,
                "success": True,
                "skipped": True,
                "reason": "duplicate",
                "duration_seconds": duration_seconds,
                "payload": payload,
            }

        result = self.request_recovery(payload)

        if result.get("success"):
            self.requested_ranges.add(request_key)
```

```

        result["duration_seconds"] = duration_seconds
        result["payload"] = payload
        return result

def build_payload(self, start_time, end_time):
    start = self._format_time(start_time)
    end = self._format_time(end_time)

    return {
        "start": start,
        "end": end,
        "start_time": start,
        "end_time": end,
    }

def request_recovery(self, payload):
    try:
        import requests

        response = requests.get(
            self.server_url,
            params={
                "start": payload["start"],
                "end": payload["end"],
            },
            timeout=self.request_timeout,
        )
    except Exception as e:
        return {
            "requested": True,
            "success": False,
            "error": str(e),
        }

    if response.status_code == 404:
        return {
            "requested": True,
            "success": False,
            "status_code": 404,
            "reason": "not_found",
            "error": "요청한 시간 구간에 해당하는 백업 파일이 없습니다.",
        }

    if not response.ok:
        return {
            "requested": True,
            "success": False,
            "status_code": response.status_code,
            "error": response.text[:200],
        }

    if not response.content:
        return {
            "requested": True,
            "success": False,
            "status_code": response.status_code,
            "error": "서버 응답에 복구 영상 파일 데이터가 없습니다.",
        }

    save_path = self._save_file_response(response, payload)

    if save_path is None:
        return {
            "requested": True,
            "success": False,
            "status_code": response.status_code,
            "error": "복구 영상 파일 저장 실패",
        }

    return {
        "requested": True,
        "success": True,
        "status_code": response.status_code,
        "saved_file": True,
        "file_path": save_path,
        "message": "복구 영상 ZIP 파일 저장 완료",
    }

def _save_file_response(self, response, payload):
    os.makedirs(self.recovery_dir, exist_ok=True)

    filename = self._get_response_filename(response)
    if not filename:
        filename = self._make_default_filename(payload)

    save_path = self._get_unique_save_path(filename)

    try:
        with open(save_path, "wb") as file:
            file.write(response.content)
    except Exception:
        return None

    return save_path

def _get_response_filename(self, response):

```

```

content_disposition = response.headers.get("Content-Disposition", "")

for part in content_disposition.split(";"):
    part = part.strip()
    lower_part = part.lower()

    if lower_part.startswith("filename*="):
        filename = part.split("=", 1)[1].strip().strip('\'')

        if filename.lower().startswith("utf-8'"):
            filename = filename[7:]

        return self._sanitize_filename(unquote(filename))

    if lower_part.startswith("filename="):
        filename = part.split("=", 1)[1].strip().strip('\'')
        return self._sanitize_filename(unquote(filename))

return None

def _make_default_filename(self, payload):
    start_time = payload["start"].replace(":", "-")
    end_time = payload["end"].replace(":", "-")

    return self._sanitize_filename(
        f"recovered_backups_{self.camera_id}_{start_time}_{end_time}.zip"
    )

def _get_unique_save_path(self, filename):
    save_path = os.path.join(self.recovery_dir, filename)

    if not os.path.exists(save_path):
        return save_path

    name, ext = os.path.splitext(filename)
    index = 2

    while True:
        candidate = os.path.join(self.recovery_dir, f"{name}_{index}{ext}")

        if not os.path.exists(candidate):
            return candidate

        index += 1

def _get_request_key(self, payload):
    return (
        self.camera_id,
        payload["start"],
        payload["end"],
    )

def _format_time(self, value):
    return value.replace(microsecond=0).isoformat()

def _sanitize_filename(self, filename):
    filename = os.path.basename(filename)
    return re.sub(r'[\<:\>\/\|?]*', "_", filename)

```

클래스 역할

NetworkRecoveryManager는 RTSP 영상 수신 중 네트워크 장애가 발생했을 때 장애 시작 시각과 복구 시각을 기록하고, 복구 후 해당 누락 시간 구간의 백업 영상을 서버에 요청하는 클라이언트 측 복구 관리 모듈이다.

이 모듈은 RTSP 영상을 직접 수신하거나 GUI를 직접 제어하지 않는다. 외부 코드에서 장애 감지 시 record_failure()를 호출하고, 복구 감지 시 record_recovery()를 호출하여 복구 요청 결과를 dict 형태로 받는 구조이다.

- 장애 발생 시점 기록
- 복구 시점 기록 및 누락 구간 계산
- GET /recover API 요청 생성
- 성공 시 recovered_backups.zip 파일 저장
- 404 또는 서버 연결 실패 시 실패 결과 반환
- 너무 짧은 장애 구간 및 중복 요청 방지

확인 및 주의사항

항목	확인 결과
서버 규격 반영	GET 방식, /recover 엔드포인트, start/end 파라미터 형식을 반영함
응답 형식	200 OK 응답의 ZIP 이진 데이터를 복구 영상 폴더에 저장하도록 처리함
404 처리	요청 구간에 해당하는 백업 파일이 없는 경우 not_found 결과를 반환함
실제 실행 시 주의	기본 server_url의 라즈베리파이IP 부분은 실제 라즈베리파이 IP로 교체해야 함
모듈 범위	서버에서 .ts 파일을 찾거나 ZIP으로 묶어주는 기능은 이 모듈의 역할이 아님

API 요청 규격

구분	내용
서버 주소	http://<라즈베리파이_IP>:8002
엔드포인트	/recover
HTTP Method	GET
요청 파라미터	start, end
시간 형식	ISO 8601 예: 2026-05-30T21:00:15
성공 응답	recovered_backups.zip 이진 데이터
조건 불일치	404 Not Found

요청 예시

GET http://<라즈베리파이_IP>:8002/recover?start=2026-05-30T21:00:15&end=2026-05-30T21:00:25

Input / Output

구분	내용
초기 설정 입력	camera_id, server_url, recovery_dir, min_failure_seconds, request_timeout
장애 감지 입력	record_failure() 호출 시점. 일반적으로 ret == False가 처음 감지된 시점
복구 감지 입력	record_recovery() 호출 시점. 일반적으로 ret == True로 다시 복구된 시점
서버 요청 값	start, end 파라미터. 둘 다 ISO 8601 문자열
성공 출력	success=True, status_code=200, saved_file=True, file_path 포함
실패 출력	success=False, error 또는 reason 포함

각 함수 역할

함수명	역할
has_active_failure()	현재 장애가 기록된 상태인지 확인한다. failure_start_time이 존재하면 True를 반환한다.
record_failure()	장애가 처음 발생한 시간을 기록한다. 이미 장애가 진행 중이면 시작 시간을 덮어쓰지 않는다.
record_recovery()	복구 시점을 기준으로 장애 지속 시간을 계산하고, 조건에 맞으면 복구 서버에 누락 구간을 요청한다.
build_payload()	start, end 값을 ISO 형식으로 만들어 GET 요청에 사용할 payload를 구성한다.
request_recovery()	requests.get()으로 /recover API에 start/end 파라미터를 보내고, 응답 결과를 dict로 반환한다.
_save_file_response()	서버가 보낸 ZIP 파일 데이터를 복구 영상 폴더에 저장한다.
_get_response_filename()	Content-Disposition 헤더에서 서버가 지정한 파일명을 추출한다.
_make_default_filename()	서버가 파일명을 주지 않았을 때 기본 ZIP 파일명을 생성한다.
_get_unique_save_path()	같은 파일명이 있을 경우 _2, _3을 붙여 덮어쓰기를 방지한다.
_get_request_key()	camera_id, start, end를 묶어 중복 요청 확인용 key를 만든다.
_format_time()	datetime 값을 microsecond 없이 ISO 8601 문자열로 바꾼다.
_sanitize_filename()	파일명으로 사용할 수 없는 특수문자를 _로 치환한다.

외부 파일 사용 예시

```

from network_recovery_manager import NetworkRecoveryManager
import time

manager = NetworkRecoveryManager(
    server_url="http://192.168.99.200:8002/recover",
    camera_id="cam01"
)

print("네트워크 장애 발생")
manager.record_failure()

time.sleep(5)

if manager.has_active_failure():
    print("네트워크 복구 감지, 복구 요청 시도")
    result = manager.record_recovery()

    if result.get("success"):
        if result.get("skipped"):
            print("복구 요청 생략:", result.get("reason"))
        else:
            print("복구 요청 성공:", result.get("file_path"))
    else:
        print("복구 요청 실패:", result.get("error"))

```

클래스 전체 흐름도

네트워크 연결 실패 또는 프레임 수신 실패 감지
↓
record_failure() 호출
↓
장애 시작 시각 저장
↓
네트워크 복구 감지
↓
record_recovery() 호출
↓
장애 시작 시각 ~ 복구 시각 계산
↓
장애 시간이 너무 짧으면 요청 생략
↓
GET /recover?start=...&end=... 요청
↓
200 OK: recovered_backups.zip 저장
404 Not Found: 해당 구간 백업 없음으로 처리
기타 오류: 실패 결과 dict 반환

성공 / 실패 반환 예시

성공 예시

```
{
  "requested": True,
  "success": True,
  "status_code": 200,
  "saved_file": True,
  "file_path": "...\복구 영상\recovered_backups.zip",
  "message": "복구 영상 ZIP 파일 저장 완료",
  "duration_seconds": 10.0,
  "payload": {
    "start": "2026-05-30T21:00:15",
    "end": "2026-05-30T21:00:25"
  }
}
```

404 실패 예시

```
{
  "requested": True,
  "success": False,
  "status_code": 404,
  "reason": "not_found",
  "error": "요청한 시간 구간에 해당하는 백업 파일이 없습니다.",
  "duration_seconds": 10.0,
  "payload": {
    "start": "2026-05-30T21:00:15",
    "end": "2026-05-30T21:00:25"
  }
}
```

테스트 방법

단계	명령 또는 확인 내용
1. 문법 검사	python -m py_compile network_recovery_manager.py
2. 장애 기록 확인	record_failure() 호출 후 failure_start_time이 기록되는지 확인
3. 복구 요청 확인	record_recovery() 호출 시 GET /recover 요청이 발생하는지 확인
4. ZIP 저장 확인	200 OK 응답 후 복구 영상 폴더에 recovered_backups.zip이 생성되는지 확인
5. 404 처리 확인	해당 구간 파일이 없을 때 success=False, reason=not_found가 반환되는지 확인